HPC Assignment

Parallel Sieve of Eratosthenes:
Domain vs. Functional Decomposition

Student Name: Sean Balbale
Course: CPSC 375: High-Performance Computing
Instructor: Peter Yoon
Date: April 8, 2026

1 Introduction

The Sieve of Eratosthenes is an ancient algorithm that identifies all prime numbers up to a specified limit n by iteratively marking the multiples of discovered primes as composite. In a parallel environment, domain decomposition divides data into pieces and associates computational steps with that data. This report analyzes the efficiency of an optimized domain decomposition model compared to a functional decomposition model using MPI in C. Benchmarks for $n = 100,000,000$ evaluate scalability, cache efficiency, and network communication overhead on both the Pine cluster and a secondary personal cluster.

2 Implementation & Optimization

2.1 Incremental Domain Decomposition (`domainSieve.c`)

The domain decomposition model relies heavily on dividing the total array of elements into p distinct, continuous blocks utilizing the standard block decomposition macros (`BLOCK_LOW`, `BLOCK_HIGH`, `BLOCK_SIZE`). To maximize efficiency, three critical architectural optimizations were implemented:

- **Eliminate Even Integers**: Memory requirements were immediately halved by stripping all even numbers out of the primary array mapping. The size calculation $elements = (n-1)/2$ shifts the index logic, meaning index i corresponds to the integer $3 + 2i$. While this slightly complicates the calculation of the first multiple (handled via a conditional parity check during sieving), it allows the algorithm to push further into the bounds of physical RAM limit.
- **Eliminate Broadcasts**: A naive MPI implementation would designate Process 0 to find a prime and broadcast it to all other nodes. Instead, every single process independently creates its own internal `base_primes` array up to $\sqrt{n}$. Because sieving up to $\sqrt{n}$ is computationally trivial, duplicating this work across all nodes entirely eliminates the massive network bottleneck of sequential `MPI_Bcast` commands.
- **Cache-Friendly Reorganized Loops**: The most impactful optimization centers on the memory hierarchy. Rather than iterating through every prime factor across the entire local array at once (which leads to catastrophic cache thrashing on large datasets), the inner and outer loops were inverted and blocked. The local array is broken down into chunks matching the L1 cache size (`CACHE_L1_SIZE 32768`). The algorithm fully sieves one cache-sized block with *all* available primes before moving to the next. This ensures spatial locality and maximizes cache hits.

2.2 Functional Decomposition (`funcSieve.c`)

Functional decomposition completely alters the parallel logic; rather than splitting the array space among nodes, it splits the *tasks* (specific prime factors).

- Every process is forced to allocate a full-sized array up to n (though still halved for odd-only representation).
- After computing the base primes up to $\sqrt{n}$, the algorithm distributes the work using a Round-Robin assignment: `if (prime_index % p == id)`. This ensures that if Process 0 handles sieving multiples of 3, Process 1 handles multiples of 5, Process 2 handles multiples of 7, etc.
- Each process sweeps the entire full-sized array marking only its assigned multiples.
- To merge the findings, the algorithm utilizes `MPI_Reduce` with a bitwise OR operator (`MPI_BOR`). This combines all the individually marked arrays across the network into one final boolean array on Process 0, where the remaining unmarked indices are tallied.

3 Benchmarking Results

Benchmarks were recorded for $n = 100,000,000$ (finding exactly 5,761,455 primes). The models were tested on both the primary Pine cluster (`submitSieve.sh`) and a personal dual-node compute cluster (`submit.sh`), scaling from 1 to 4 processors using OpenMPI/PMI2 over TCP.

Table 1: Pine Cluster Benchmark Data

Processors	**Domain Time (s)**	**Functional Time (s)**
1	0.450 605	0.796 236
2	0.241 571	0.496 460
4	0.121 870	0.418 110

Table 2: Personal Cluster Benchmark Data

Processors	**Domain Time (s)**	**Functional Time (s)**
1	0.070 812	0.321 143
2	0.037 895	0.336 463
4	0.020 315	0.590 287

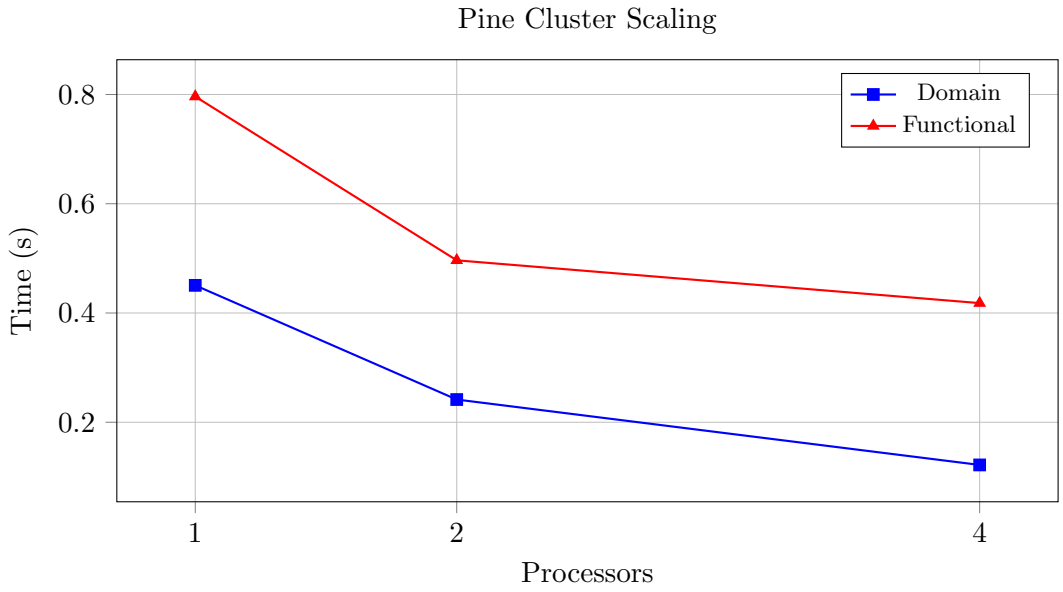

Figure 1: Execution time comparison on the Pine cluster. Domain decomposition achieves near-linear speedup.

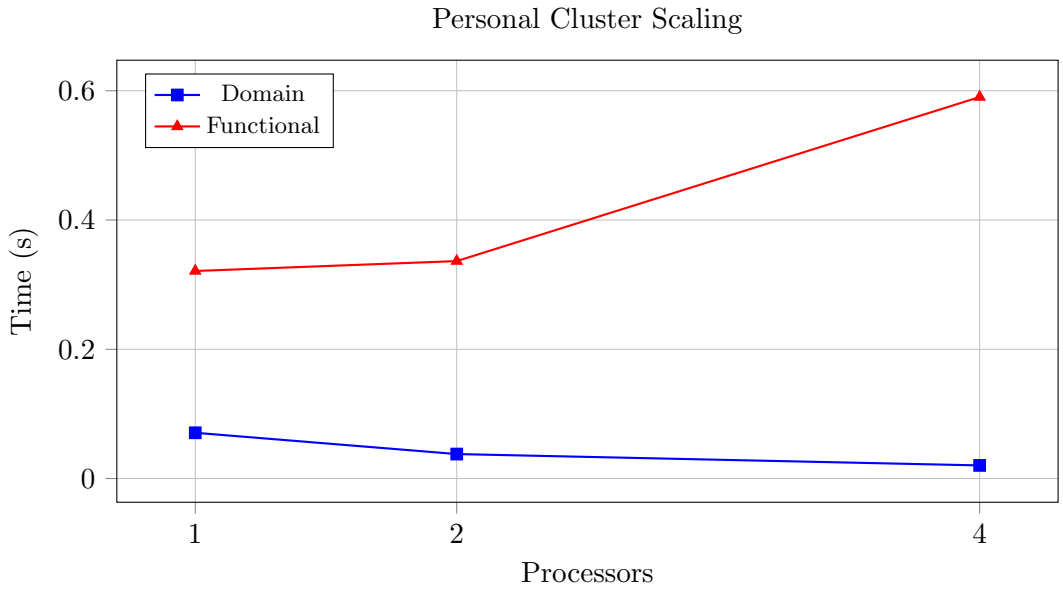

Figure 2: Execution time comparison on the Personal cluster. Functional decomposition actively slows down as nodes are added.

4 Performance Analysis

4.1 Domain Scalability & Cache Coherency

The domain decomposition model (`domainSieve.c`) demonstrates excellent, near-linear strong scaling. On the Pine cluster, moving from 1 to 4 processors dropped the time from 0.45s to 0.12s (a 3.7× speedup). On the faster personal cluster, it dropped from 0.07s to 0.02s (a 3.48× speedup). By isolating both the memory boundaries and the communication steps, the processors are completely unimpeded during the calculation phase. The cache optimization ensures that the CPU never stalls waiting for RAM fetches, and the single `MPI_Reduce` containing only one `long long` integer per node at the very end of the program keeps network overhead virtually nonexistent.

4.2 Functional Failure & Network Bottlenecks

The functional decomposition model (`funcSieve.c`) exposes the severe limitations of poorly optimized parallel data distribution. While it slightly scales on the Pine cluster (dropping from 0.79s to 0.41s), it experiences catastrophic negative scaling on the personal cluster, where adding 4 processors actively *slows down* the execution time to 0.59s compared to a single processor running at 0.32s.

This degradation occurs for two structural reasons:

1. **Redundant Memory Load:** Because every process requires a full copy of the n-sized array, running multiple tasks on the same node (e.g., `--ntasks-per-node=2`) starves the CPU of memory bandwidth and evicts other processes from the L3 cache.
2. **Massive Reduction Overhead:** Unlike the domain model which sends a single integer over the network, the functional model must merge the entire 50MB array using `MPI_Reduce` with a bitwise OR operation. Because the personal cluster relies on standard TCP over a 10.0.0.0/24 interface (as seen in `submit.sh`), the network card becomes fully saturated trying to synchronize tens of megabytes of data across the distributed nodes, entirely eclipsing the time saved by splitting the mathematical workload.

5 Conclusion

The data conclusively proves that domain decomposition is the superior architectural model for the Sieve of Eratosthenes. By prioritizing cache-friendly array blocking and eliminating inter-process communication during the sieving phase, the algorithm scales almost perfectly with hardware. Functional decomposition, while conceptually interesting for task distribution, is critically hamstrung by its redundant memory allocation and the heavy bandwidth penalty of synchronizing massive data structures across the network.

A Source Code: Domain Decomposition (`domainSieve.c`)

```c
#include <mpi.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

// Block Decomposition Macros
#define BLOCK_LOW(id,p,n)  ((id)*(n)/(p))
#define BLOCK_HIGH(id,p,n) (BLOCK_LOW((id)+1,p,n)-1)
#define BLOCK_SIZE(id,p,n) (BLOCK_LOW((id)+1,p,n)-BLOCK_LOW(id,p,n))
#define BLOCK_OWNER(index,p,n) (((p)*(index)+1)-1)/(n)

#define CACHE_L1_SIZE 32768 // Typical 32KB L1 cache size

int main(int argc, char *argv[]) {
    int id, p;
    long long n, size, elements, i, j, k;
    long long low_value, high_value;
    char *marked;
    long long global_count = 0, local_count = 0;
    double elapsed_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    if (argc != 2) {
        if (!id) printf("Usage: %s <n>\n", argv[0]);
        MPI_Finalize();
        exit(1);
    }

    n = atoll(argv[1]);

    // Optimization 1: Eliminate Even Integers. Only represent odd numbers from 3 to n.
    // Total odd numbers to check = (n - 1) / 2
    elements = (n - 1) / 2;

    low_value = 3 + 2 * BLOCK_LOW(id, p, elements);
    high_value = 3 + 2 * BLOCK_HIGH(id, p, elements);
    size = BLOCK_SIZE(id, p, elements);

    marked = (char *)calloc(size, sizeof(char));
    if (marked == NULL) {
        printf("Process %d: Cannot allocate memory\n", id);
        MPI_Finalize();
        exit(1);
    }

    MPI_Barrier(MPI_COMM_WORLD);
    elapsed_time = -MPI_Wtime();

    // Optimization 2: Eliminate Broadcasts.
    // Each process independently finds primes up to sqrt(n)
    long long sqrt_n = (long long)sqrt((double)n);
    long long sqrt_elements = (sqrt_n - 1) / 2;
```

```
    char *base_primes = (char *)calloc(sqrt_elements + 1, sizeof(char));

    for (i = 0; i <= sqrt_elements; i++) {
        if (!base_primes[i]) {
            long long prime = 3 + 2 * i;
            for (j = i + prime; j <= sqrt_elements; j += prime) {
                base_primes[j] = 1;
            }
        }
    }

    // Optimization 3: Reorganize Loops for Cache Efficiency (Block Sieving)
    // Iterate through our local array in chunks that fit comfortably in the L1
    cache.
    long long block_size = CACHE_L1_SIZE;
    long long num_blocks = (size + block_size - 1) / block_size;

    for (long long b = 0; b < num_blocks; b++) {
        long long block_start_idx = b * block_size;
        long long block_end_idx = block_start_idx + block_size - 1;
        if (block_end_idx >= size) block_end_idx = size - 1;

        long long block_low_val = low_value + 2 * block_start_idx;
        long long block_high_val = low_value + 2 * block_end_idx;

        // Iterate over all base primes for this specific cache-sized block
        for (i = 0; i <= sqrt_elements; i++) {
            if (!base_primes[i]) {
                long long prime = 3 + 2 * i;
                long long first;

                // Find the first multiple of the prime within this block
                if (prime * prime > block_low_val) {
                    first = (prime * prime - low_value) / 2;
                } else {
                    long long rem = block_low_val % prime;
                    if (rem == 0) {
                        first = block_start_idx;
                    } else {
                        // Math to align with odd multiples
                        long long multiple = block_low_val + (prime - rem);
                        if (multiple % 2 == 0) multiple += prime;
                        first = (multiple - low_value) / 2;
                    }
                }

                // Sieve the current block
                for (j = first; j <= block_end_idx; j += prime) {
                    marked[j] = 1;
                }
            }
        }
    }

    // Count primes in local domain
    for (i = 0; i < size; i++) {
        if (!marked[i]) local_count++;
    }

```

```
    // Combine counts.
    MPI_Reduce(&local_count, &global_count, 1, MPI_LONG_LONG, MPI_SUM, 0,
    MPI_COMM_WORLD);

    elapsed_time += MPI_Wtime();

    if (!id) {
        // Add 1 to account for the only even prime, 2.
        printf("%lld primes are less than or equal to %lld\n", global_count + 1, n
    );
        printf("Total elapsed time: %10.6f seconds\n", elapsed_time);
    }

    free(marked);
    free(base_primes);
    MPI_Finalize();
    return 0;
}
```

B Source Code: Functional Decomposition (`funcSieve.c`)

```c
#include <mpi.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int id, p;
    long long n, i, j;
    char *local_marked;
    char *global_marked = NULL;
    long long global_count = 0;
    double elapsed_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    if (argc != 2) {
        if (!id) printf("Usage: %s <n>\n", argv[0]);
        MPI_Finalize();
        exit(1);
    }

    n = atoll(argv[1]);

    // Each process allocates an array representing odd numbers up to n.
    // (This still halves the memory, but every process holds a full copy of the bounds).
    long long elements = (n - 1) / 2;
    local_marked = (char *)calloc(elements, sizeof(char));

    if (id == 0) {
        global_marked = (char *)calloc(elements, sizeof(char));
    }

    MPI_Barrier(MPI_COMM_WORLD);
    elapsed_time = -MPI_Wtime();

    // Step 1: Every process finds base primes up to sqrt(n)
    long long sqrt_n = (long long)sqrt((double)n);
    long long sqrt_elements = (sqrt_n - 1) / 2;
    char *base_primes = (char *)calloc(sqrt_elements + 1, sizeof(char));

    for (i = 0; i <= sqrt_elements; i++) {
        if (!base_primes[i]) {
            long long prime = 3 + 2 * i;
            for (j = i + prime; j <= sqrt_elements; j += prime) {
                base_primes[j] = 1;
            }
        }
    }

    // Step 2: Functional Decomposition
    // Distribute the primes to processes using Round-Robin
    int prime_index = 0;
    for (i = 0; i <= sqrt_elements; i++) {
```

```
        if (!base_primes[i]) {
            long long prime = 3 + 2 * i;

            // Only process 'id' handles this prime
            if (prime_index % p == id) {
                long long first_idx = (prime * prime - 3) / 2;
                // Sieve the entire full-sized local array for this prime
                for (j = first_idx; j < elements; j += prime) {
                    local_marked[j] = 1;
                }
            }
            prime_index++;
        }
    }

    // Step 3: Merge all distributed full-size arrays into Process 0 using Logical OR
    MPI_Reduce(local_marked, global_marked, elements, MPI_CHAR, MPI_BOR, 0, MPI_COMM_WORLD);

    elapsed_time += MPI_Wtime();

    // Step 4: Process 0 counts the final unmarked elements
    if (id == 0) {
        for (i = 0; i < elements; i++) {
            if (!global_marked[i]) global_count++;
        }
        printf("%lld primes are less than or equal to %lld\n", global_count + 1, n);
        printf("Total elapsed time: %10.6f seconds\n", elapsed_time);
        free(global_marked);
    }

    free(local_marked);
    free(base_primes);
    MPI_Finalize();
    return 0;
}
```

C Submit Script: Pine Cluster (`submitSieve.sh`)

```
#!/bin/bash
#SBATCH --job-name=pine_sieve_benchmark
#SBATCH --output=pine_sieve_results_%j.txt
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=2
#SBATCH --time=00:15:00
#SBATCH --partition=defq

module load openmpi

echo "--- Running Domain Decomposition ---"
echo "1 Processor:"
srun --ntasks=1 ./domainSieve 100000000
echo "2 Processors:"
srun --ntasks=2 ./domainSieve 100000000
echo "4 Processors:"
srun --ntasks=4 ./domainSieve 100000000

echo "--- Running Functional Decomposition ---"
echo "1 Processor:"
srun --ntasks=1 ./funcSieve 100000000
echo "2 Processors:"
srun --ntasks=2 ./funcSieve 100000000
echo "4 Processors:"
srun --ntasks=4 ./funcSieve 100000000
```

D Submit Script: Personal Cluster (`submit.sh`)

```
#!/bin/bash
#SBATCH --job-name=sieve_benchmark
#SBATCH --partition=compute
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=2
#SBATCH --output=sieve_results.out
#SBATCH --error=sieve_results.err

# 1. Environment Setup (Replaces the missing 'module' command)
source /opt/intel/oneapi/setvars.sh > /dev/null 2>&1

# 2. Network & MPI Handshake Fixes (Specific to your cluster)
export FI_PROVIDER=tcp
export FI_TCP_IFACE=10.0.0.0/24
export I_MPI_HYDRA_BOOTSTRAP=slurm
export I_MPI_PMI_LIBRARY=/usr/lib64/libpmi2.so.0

# 3. Execution
echo "======================================================="
echo " Sieve of Eratosthenes Benchmarks"
echo " Date: $(date)"
echo "======================================================="
echo ""

# Note: The assignment asks for 1, 2, and 4 processors.
# We adjust -N (nodes) and -n (tasks/processors) to scale up.

echo "--- Running Domain Decomposition ---"
echo "1 Processor:"
srun --mpi=pmi2 -N 1 -n 1 ./domainSieve 100000000
echo "2 Processors:"
srun --mpi=pmi2 -N 1 -n 2 ./domainSieve 100000000
echo "4 Processors:"
srun --mpi=pmi2 -N 2 -n 4 ./domainSieve 100000000
echo ""

echo "--- Running Functional Decomposition ---"
echo "1 Processor:"
srun --mpi=pmi2 -N 1 -n 1 ./funcSieve 100000000
echo "2 Processors:"
srun --mpi=pmi2 -N 1 -n 2 ./funcSieve 100000000
echo "4 Processors:"
srun --mpi=pmi2 -N 2 -n 4 ./funcSieve 100000000
echo ""
echo "Benchmarks Complete!"
```

E Raw Results: Pine Cluster

```
--- Running Domain Decomposition ---
1 Processor:
srun: warning: can't run 1 processes on 2 nodes, setting nnodes to 1
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.450605 seconds
2 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.241571 seconds
4 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.121870 seconds
--- Running Functional Decomposition ---
1 Processor:
srun: warning: can't run 1 processes on 2 nodes, setting nnodes to 1
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.796236 seconds
2 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.496460 seconds
4 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.418110 seconds
```

F Raw Results: Personal Cluster

```
=====================================================
 Sieve of Eratosthenes Benchmarks
 Date: Wed Apr  8 07:09:44 PM EDT 2026
=====================================================

--- Running Domain Decomposition ---
1 Processor:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.070812 seconds
2 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.037895 seconds
4 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.020315 seconds

--- Running Functional Decomposition ---
1 Processor:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.321143 seconds
2 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.336463 seconds
4 Processors:
5761455 primes are less than or equal to 100000000
Total elapsed time:   0.590287 seconds

Benchmarks Complete!
```